Architecture et Systèmes d'Exploitation : Gestion des Processus, Cycle de Vie et Signaux POSIX

Thème principal : Gestion des Processus et Signaux POSIX sous Unix/Linux

Synthèse exécutive : Ce cours aborde l'architecture interne des systèmes d'exploitation de type Unix/Linux en traitant du concept fondamental de processus. Il explicite la transition d'un programme statique stocké sur disque vers un processus dynamique s'exécutant en mémoire RAM. Pour faire fonctionner de nombreuses applications simultanément sur une machine, le noyau s'appuie sur le multitâche préemptif et orchestre le cycle de vie des processus à travers trois états fondamentaux (Élu, Prêt, Bloqué). Le cours détaille la structure de contrôle (PCB), la filiation arborescente parent-enfant (PID/PPID) issue du processus initial systemd/init (PID 1), les mécanismes de création via fork() et exec(), ainsi que le système universel de communication et de contrôle par signaux POSIX (SIGINT, SIGTERM, SIGKILL, SIGSTOP, SIGCONT).

Du Programme Statique au Processus Dynamique : Cycle de Vie, Attributs et Multitâche

Pour exécuter une application informatique, le système d'exploitation doit faire basculer le programme (suite d'instructions statique enregistrée sur le stockage secondaire) vers un état dynamique appelé processus en le déversant dans la mémoire vive (RAM). Dans un système Unix/Linux, la gestion du processeur s'appuie sur le multitâche préemptif. L'ordonnanceur (scheduler) du noyau découpe le temps d'exécution en tranches ultra-courtes appelées quanta. Il gère le passage dynamique de chaque processus à travers un modèle strict à trois états : 1. Élu (Running) : Le processus est pris en charge par un cœur du processeur et exécute activement ses instructions machine. 2. Prêt (Ready) : Le processus est intégralement chargé en RAM avec toutes ses ressources, mais attend que l'ordonnanceur lui attribue son prochain quantum de temps CPU. 3. Bloqué (Waiting / Blocked) : Le processus requiert une attente externe (lecture de fichier sur disque, paquet réseau, saisie clavier). Le noyau libère immédiatement le CPU pour une autre tâche et place le processus en attente. Lorsque l'événement survient, la donnée devient disponible : le processus ne repasse JAMAIS directement à l'état Élu, mais est réinséré dans la file d'attente à l'état Prêt.

- • Programme vs Processus : Un programme est un fichier binaire passif résidant sur le disque dur ; un processus est une instance dynamique active chargée en mémoire RAM consommant des ressources CPU et mémoire.
- • Multitâche Préemptif & Ordonnanceur : Le noyau (kernel) alloue de très courts quanta de temps processeur aux processus éligibles pour simuler une exécution simultanée.
- • Process Control Block (PCB) : Structure de données interne au noyau regroupant les métadonnées de chaque processus (état du processeur, registres, carte mémoire, PID, PPID, UID, descripteurs de fichiers).
- • Cycle de Vie à 3 États : Élu (Running - exécute son code sur le CPU), Prêt (Ready - dispose de ses ressources RAM et attend du temps CPU), Bloqué (Waiting/Blocked - suspendu en attente d'une opération d'Entrée/Sortie ou d'un événement externe).

Cas concret enseignant : *L'analogie du livre de recettes de cuisine : Le programme stocké sur le disque dur est comparé à un livre de recettes sagement rangé dans une bibliothèque (passif, statique, ne consommant ni électricité ni ingrédients). Le processus est le cuisinier en train d'exécuter activement la recette dans sa cuisine (dynamique, occupant de la place en RAM sur le plan de travail, consommant du temps CPU lors de la cuisson et manipulant des ustensiles équivalant aux descripteurs de fichiers).*

Filiation, Arborescence et Mécanismes de Création (fork et exec)

Sous Unix, aucun processus ne peut apparaître de façon isolée ou spontanée. Tous les processus s'inscrivent dans un arbre généalogique strict. Lors du démarrage du système d'exploitation, le noyau initialise le premier processus utilisateur nommé systemd (ou init), recevant impérativement le PID 1. Lorsqu'un utilisateur lance une commande

dans une console Bash (comme 'ls' ou 'vim'), le shell interactif (processus parent) exécute la création de la tâche en deux temps : 1. fork() : Le parent invoque l'appel système fork(). Le noyau duplique alors le processus parent. Le fils hérite d'une copie conforme de l'espace mémoire du père, reçoit un nouveau PID unique et prend le PID du père comme PPID. 2. exec() : Le processus fils invoque immédiatement l'appel système exec(). Le noyau remplace l'image mémoire du fils par le code binaire du nouveau programme à exécuter. Le processus conserve son PID mais exécute désormais les instructions du nouveau programme.

- • Identifiants PID et PPID : Chaque processus possède un identifiant unique (Process ID) et fait référence à l'identifiant de son processus parent (Parent Process ID).
- • Processus Racine PID 1 : Tout processus dérive d'une arborescence généalogique unique remontant jusqu'au premier processus utilisateur systemd ou init (PID 1).
- • Appel Système fork() : Duplique à l'identique le processus parent pour engendrer un processus enfant avec son propre PID.
- • Appel Système exec() : Remplace le code et l'espace mémoire d'un processus par un nouveau programme binaire à exécuter.

Contrôle des Tâches et Communication Inter-Processus : Les Signaux POSIX

Les signaux POSIX représentent la forme de communication inter-processus (IPC) la plus fondamentale dans les systèmes Unix. Lorsqu'un signal est adressé à un processus cible, le noyau interrompt immédiatement le déroulement normal de ses instructions pour traiter le signal. À la réception d'un signal, un programme dispose de trois options : 1. Exécuter l'action par défaut du système (souvent la terminaison du processus). 2. Ignorer le signal (ignore). 3. Intercepter le signal (catch) en liant l'événement à un gestionnaire de signal (signal handler). Cela permet par exemple à un éditeur de texte d'intercepter SIGTERM pour afficher 'Voulez-vous enregistrer vos modifications ?' avant de fermer. Toutefois, la spécification POSIX réserve une autorité absolue au système d'exploitation et à l'administrateur : les signaux SIGKILL (9) et SIGSTOP (19) sont gérés directement et exclusivement par le noyau. Aucun programme utilisateur ne peut intercepter ou bloquer ces deux signaux, garantissant la possibilité de neutraliser n'importe quelle tâche bloquée ou malveillante.

- • Signal POSIX : Interruption logicielle asynchrone envoyée par le noyau à un processus pour lui notifier un événement ou lui ordonner une action.
- • Gestionnaire de Signal (Signal Handler) : Routine logicielle définie par le développeur pour intercepter (catch) un signal et exécuter une action personnalisée.
- • SIGINT (Signal 2) : Émis via Ctrl + C ; demande une interruption douce d'un processus interactif.
- • SIGTERM (Signal 15) : Signal de terminaison propre envoyé par défaut par la commande kill ; autorise le nettoyage des ressources avant arrêt.
- • SIGKILL (Signal 9) : Ordre de destruction inconditionnel exécuté directement par le noyau. STRICTEMENT IMPOSSIBLE à intercepter (catch), masquer ou ignorer.
- • Job Control (SIGSTOP 19 / SIGCONT 18) : Ctrl + Z envoie SIGSTOP pour suspendre un processus ; fg ou bg envoient SIGCONT pour reprendre son exécution.

Cas concret enseignant : *L'analogie de l'expulsion du bâtiment : SIGTERM (Signal 15) est le gardien d'immeuble qui frappe poliment à votre porte pour vous demander de ranger vos affaires et d'évacuer (vous disposez du temps nécessaire pour sauvegarder vos fichiers et fermer la porte à clé). SIGKILL (Signal 9) représente l'intervention de la brigade de sécurité d'urgence qui défonce la porte et vous dégage sur-le-champ sans vous laisser la moindre seconde pour toucher à quoi que ce soit.*

■ Questions & Interventions des Étudiants en Classe

Question étudiant : Pourquoi la commande 'kill 9 1234' renvoie-t-elle une erreur dans le terminal Linux alors que 'kill -9 1234' fonctionne parfaitement ?
Réponse du professeur : La commande 'kill' attend les numéros de signaux précédés d'un tiret (ex: -9 ou -SIGKILL). Si vous tapez 'kill 9 1234' sans le tiret, l'utilitaire interprète '9' non pas comme un numéro de signal, mais comme le PID du premier processus à contacter, et '1234' comme le second. Le shell va donc tenter d'envoyer le signal par défaut SIGTERM (15) au PID 9 puis au PID 1234. Si le PID 9 n'existe pas ou ne vous appartient pas, le terminal renvoie une erreur de type 'No such process' ou 'Operation not permitted'.

Question étudiant : Qu'est-ce qu'un processus dit 'Zombie' et comment peut-on le détruire avec la commande kill ?
Réponse du professeur : Un processus Zombie est une tâche qui a terminé son exécution (état terminé/zombie) mais dont l'entrée subsiste dans la table des processus du noyau parce que son processus parent n'a pas encore lu son code de retour via l'appel système wait() ou waitpid(). Il est impossible de détruire un zombie avec 'kill -9' car le processus est DÉJÀ mort (il n'occupe plus de mémoire ni de CPU, seulement une ligne dans la table). Pour faire disparaître un zombie, il faut soit forcer son parent à lire son état, soit terminer le processus parent. Lorsque le parent s'arrête, le zombie est adopté par systemd/init (PID 1) qui nettoie immédiatement la table.

■ Exercice & Démonstration d'Application

Énoncé : Dans un terminal Bash sous Linux, un administrateur effectue la séquence de commandes suivante : 1. sleep 600 & 2. ps -ef | grep sleep 3. kill -15 Explicitez le rôle exact du symbole '&', détaillez ce qui se passe lors de l'exécution de la commande kill -15 et décrivez le résultat de la vérification par ps.

Démarche de résolution :
Étape 1 : Le symbole '&' placé à la fin de la commande 'sleep 600 &' indique au shell interactif d'exécuter la tâche en arrière-plan (background). Le shell affiche instantanément l'identifiant de job (ex: [1]) ainsi que le PID attribué par le noyau (ex: 4521) et rend immédiatement la main à l'utilisateur sans bloquer le terminal. Étape 2 : La commande 'ps -ef | grep sleep' filtre la liste globale des processus système. La sortie confirme que le processus 'sleep 600' est actif avec le PID 4521 et identifie son PPID (le PID du shell Bash hôte). Étape 3 : La commande 'kill -15 4521' transmet le signal SIGTERM (Signal 15) au processus. L'utilitaire sleep ne possédant pas de gestionnaire spécifique pour capturer SIGTERM, l'action par défaut du système s'applique : l'arrêt propre et la suppression de la tâche.

Résultat : Le processus 'sleep 600' est détruit de manière propre par le signal SIGTERM. Une vérification ultérieure au moyen de 'ps -ef | grep sleep' confirme sa disparition complète de la table des processus actifs.

■ Auto-Évaluation Flash d'Examen

Q : Quel est le nom et le PID du tout premier processus utilisateur démarré par le noyau Linux au lancement du système ?

R : Il s'agit du processus 'systemd' (ou 'init' sur les anciennes distributions), auquel est attribué le PID 1.

Q : Quel signal POSIX est généré et transmis au processus au premier plan lorsqu'un utilisateur presse la combinaison 'Ctrl + C' dans son terminal ?

R : C'est le signal SIGINT (Signal 2), qui demande l'interruption modérée de la tâche interactive.

Q : Pourquoi le signal SIGKILL (Signal 9) ne peut-il jamais être intercepté par un gestionnaire de signal écrit par un développeur ?

R : Parce que la norme POSIX impose que SIGKILL soit géré et exécuté directement par le noyau (kernel). Le signal ne parvient jamais à la couche applicative du processus, ce qui empêche tout interfaçage ou blocage.